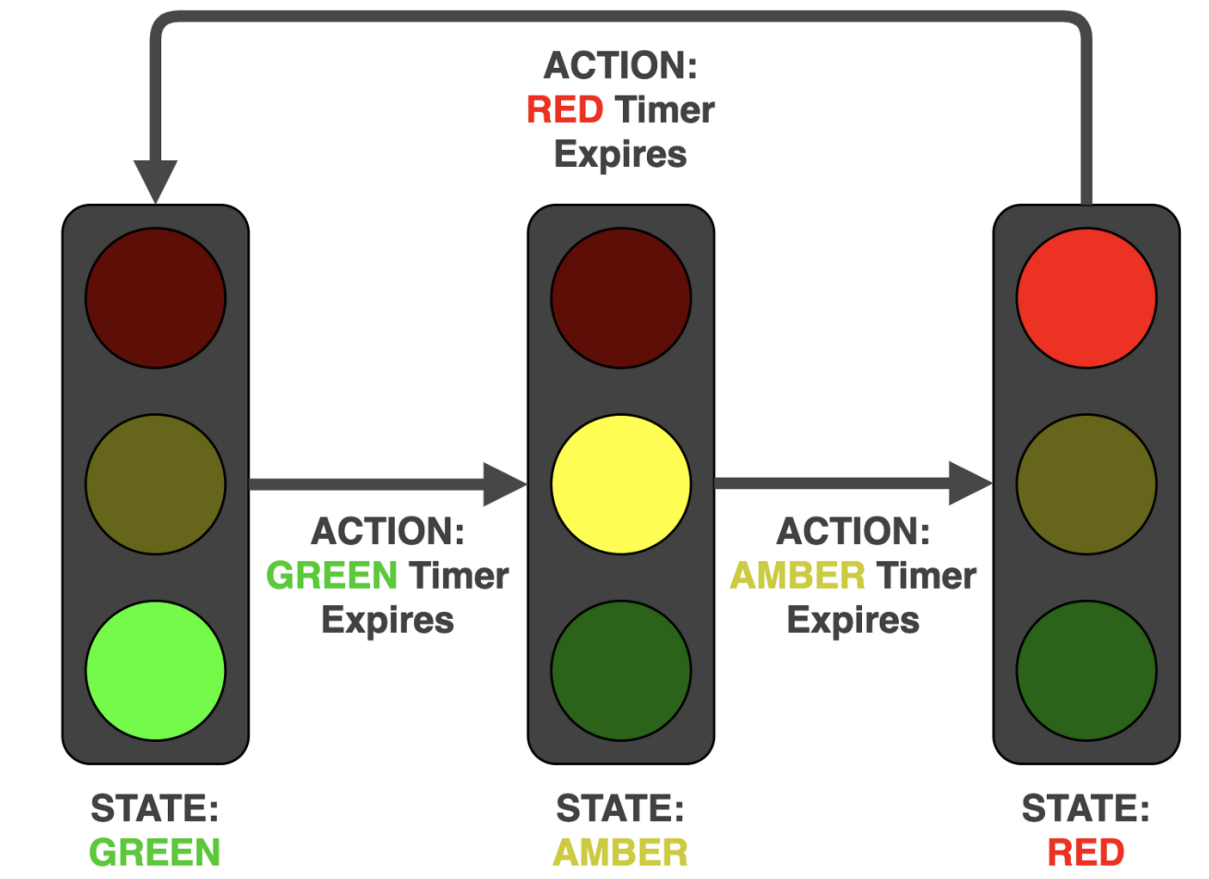


Formation FPGA :

TP n°3 : Machine à états (FSM)



Yassine Aouini

Objectifs du TP :	2
Question 1 :	2
Question 2 :	2
Question 3 :	3
Question 4 :	3
Question 5 :	4
Question 6 :	4
Question 7 :	5
Question 8 :	5
Question 9 :	6
Question 10 :	6
Question 11 :	7
Question 12 :	8

Objectifs du TP :

- Réaliser une architecture permettant de faire clignoter deux LEDs en rouge, vert et bleu.
- Utiliser des machines à états.
- Utiliser des paramètres génériques.
- Utiliser des modules.

Question 1 :

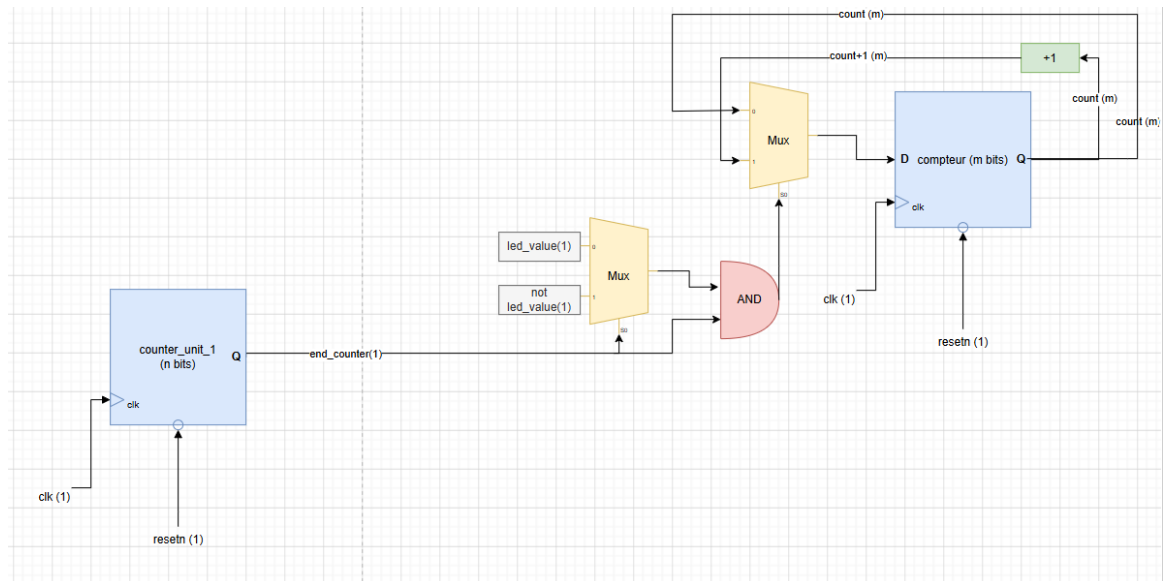
Nous allons nous servir du fichier créé dans le TP2 afin d'obtenir un module réutilisable.

L'utilisation de paramètres génériques permet d'adapter notre nouveau composant à différentes situations, sans changer son code.

Ainsi, les valeurs de simulation peuvent être réduites par gain de temps et de ressources.

Question 2 :

Voici le schéma RTL du système :



Le premier compteur correspond à notre module Counter_unit. Le second compteur va nous permettre de compter le nombre de fois que la LED s'allume puis s'éteint. Pour que cet évènement ait lieu, on va tester la condition ("la LED est éteinte" AND "le signal end_counter est émis").

Disjonction de cas :

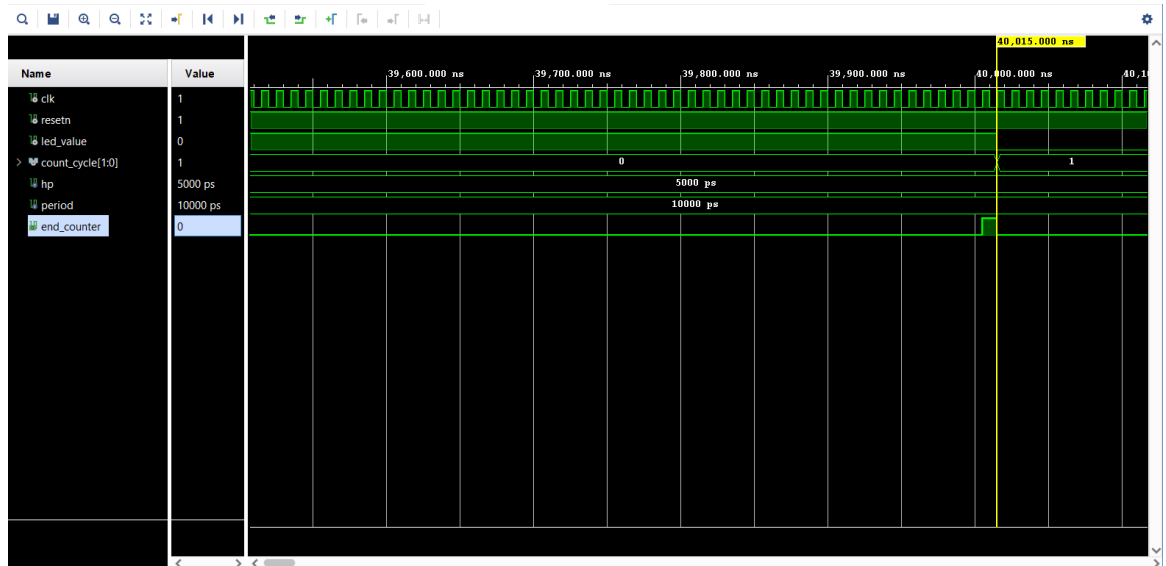
- Le bouton reset est appuyé : la valeur du compteur se remet donc à zéro
- L'évènement n'a pas eu lieu : le compteur maintient sa valeur actuelle
- L'évènement a eu lieu : le compteur s'incrémente

Question 3 :

Cf. Fichiers vhd joints avec le rapport

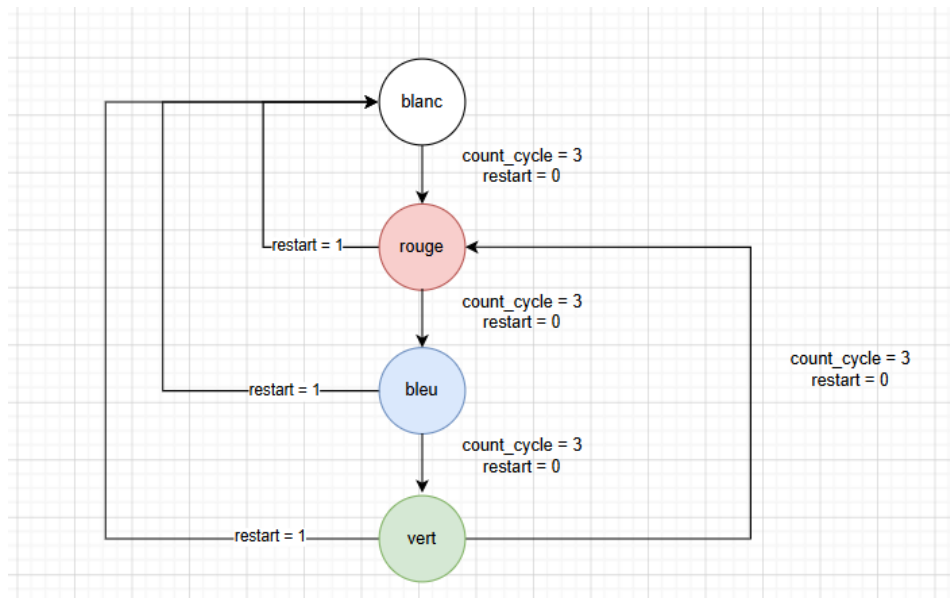
Question 4 :

On observe sur le chronogramme que le signal `end_counter` est bien reçu et que son activation permet le changement de l'état de la LED. J'ai choisi de montrer sur le chronogramme la phase d'extinction de la LED afin de montrer l'incrémentement du second compteur.



Question 5 :

Voici le schéma RTL de la machine à état :



Pour passer d'un état à l'autre, la LED doit clignoter 3 fois (`count_cycle = 3`) et le bouton `restart` ne doit pas être appuyé.

Si le bouton `restart` est appuyé, on revient à l'état initial.

Question 6 :

Nom du signal	Entrée/Sortie/Signal interne	Taille en bits
CLK	Entrée	1
Resetn	Entrée	1
Restart	Entrée	1
Led_r	Sortie	1
Led_g	Sortie	1
Led_b	Sortie	1
End_counter	Signal interne	1
Led_value	Signal interne	1
Current_state	Signal interne	1
Next_state	Signal interne	1

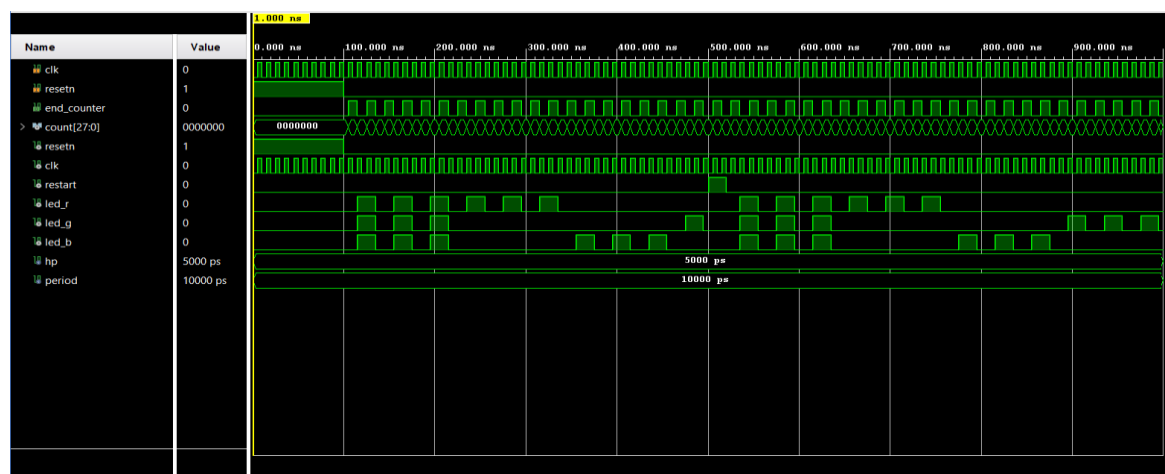
Question 7 :

Cf. Fichiers vhd joints avec le rapport

Question 8 :

Pour vérifier le fonctionnement de notre architecture, on doit s'assurer que :

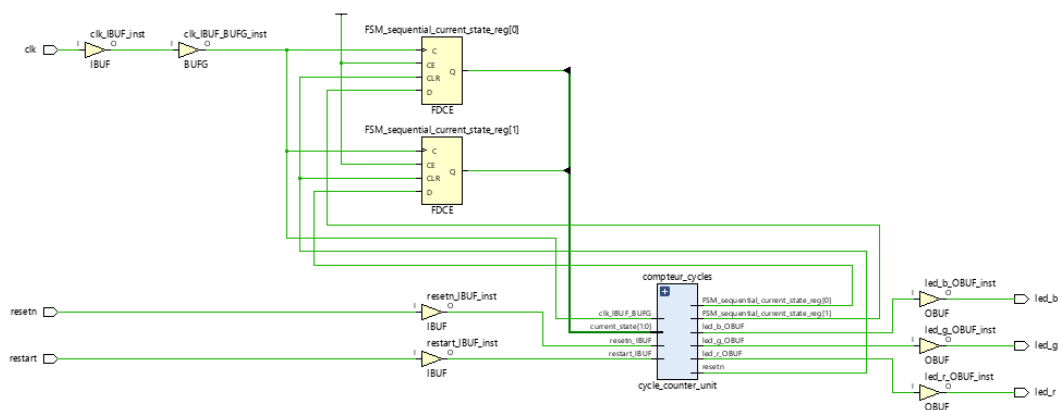
- La LED clignote 3 fois dans chaque état souhaité
- L'ordre des couleurs est respecté (blanc/rouge/bleu/vert puis rouge/bleu/vert)
- L'appui sur le bouton restart remette la LED à l'état initial
- L'appui sur le bouton restart remette à zéro le compteur de cycle



Le fonctionnement est fidèle à nos attentes : trois clignotements par état de la LED, un retour à l'état initial en cas de restart, et les couleurs s'allument dans l'ordre.

```
Note: OK : reset -> LEDs éteintes
Time: 20 ns Iteration: 0 Process: /tb_tp_fsm/line__151 File: C:/Users/yassi/Formation_FPGA_Yassine_Aouini/TP3_FPGA_Yassine/TP3_FPGA_Yassine.srcs/sources_1/new/tb_tp_fsm
Note: OK : sequence blanc/rouge/bleu/vert/rouge correcte
Time: 615 ns Iteration: 5 Process: /tb_tp_fsm/line__151 File: C:/Users/yassi/Formation_FPGA_Yassine_Aouini/TP3_FPGA_Yassine/TP3_FPGA_Yassine.srcs/sources_1/new/tb_tp_fsm
Note: OK : restart -> retour au blanc, 3 clignotements recomptes
Time: 755 ns Iteration: 5 Process: /tb_tp_fsm/line__151 File: C:/Users/yassi/Formation_FPGA_Yassine_Aouini/TP3_FPGA_Yassine/TP3_FPGA_Yassine.srcs/sources_1/new/tb_tp_fsm
Note: OK : reset en marche -> repart du blanc
Time: 885 ns Iteration: 5 Process: /tb_tp_fsm/line__151 File: C:/Users/yassi/Formation_FPGA_Yassine_Aouini/TP3_FPGA_Yassine/TP3_FPGA_Yassine.srcs/sources_1/new/tb_tp_fsm
Note: SIMULATION TERMINEE : tous les tests sont passés
Time: 885 ns Iteration: 5 Process: /tb_tp_fsm/line__151 File: C:/Users/yassi/Formation_FPGA_Yassine_Aouini/TP3_FPGA_Yassine/TP3_FPGA_Yassine.srcs/sources_1/new/tb_tp_fsm
INFO: [USF-Xsim-96] Xsim completed. Design snapshot 'tb_tp_fsm_behav' loaded.
INFO: [USF-Xsim-97] Xsim simulation ran for 1000ns
> launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:06 . Memory (MB): peak = 1588.059 ; gain = 0.000
```

Question 9 :



Le compteur de cycle est la partie bleue du Schematic.

29	
30	+-----+-----+-----+-----+-----+
31	Site Type Used Fixed Available Util%
32	+-----+-----+-----+-----+-----+
33	Slice LUTs* 29 0 17600 0.16
34	LUT as Logic 29 0 17600 0.16
35	LUT as Memory 0 0 6000 0.00
36	Slice Registers 21 0 35200 0.06
37	Register as Flip Flop 21 0 35200 0.06
38	Register as Latch 0 0 35200 0.00
39	F7 Muxes 0 0 8800 0.00
40	F8 Muxes 0 0 4400 0.00
41	+-----+-----+-----+-----+-----+

Pour un compteur de taille arbitraire 16 bits, on obtient un nombre de registres cohérents :

- 16 registres pour le compteur de temporalité
- 2 registres pour le compteur de cycles
- 1 registre pour la valeur de la LED (On/Off)
- 2 registres pour la FSM (4 états)

Question 10 :

On peut voir que dans le fichier de contrainte, la période de l'horloge est de 8ns pour une fréquence de 125MHz. Ainsi, pour passer à une fréquence de 100MHz comme demandé, on doit modifier la valeur de la période. Avec une période de 10ns, on obtient donc la bonne fréquence.

On modifie ensuite le fichier de contrainte pour adapter le nom des signaux utilisés ainsi que la nouvelle période de l'horloge trouvée.

```
1  ## This file is a general .xdc for the Cora Z7-07S Rev. B
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  # PL System Clock
7  set_property -dict { PACKAGE_PIN H16   IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L13P_T2_MRCC_35 Sch=sysclk
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 4} [get_ports { clk }];#set
9
10 # RGB LEDs
11 set_property -dict { PACKAGE_PIN L15   IOSTANDARD LVCMOS33 } [get_ports { led0_b }]; #IO_L22N_T3_AD7N_35 Sch=led0_b
12 set_property -dict { PACKAGE_PIN G17   IOSTANDARD LVCMOS33 } [get_ports { led0_g }]; #IO_L16P_T2_35 Sch=led0_g
13 set_property -dict { PACKAGE_PIN N15   IOSTANDARD LVCMOS33 } [get_ports { led0_r }]; #IO_L21P_T3_DQS_AD14P_35 Sch=led0_r
14 #set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b
15 #set_property -dict { PACKAGE_PIN L14   IOSTANDARD LVCMOS33 } [get_ports { led1_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g
16 #set_property -dict { PACKAGE_PIN M15   IOSTANDARD LVCMOS33 } [get_ports { led1_r }]; #IO_L23N_T3_35 Sch=led1_r
17
18 # Buttons
19 sset_property -dict { PACKAGE_PIN D20   IOSTANDARD LVCMOS33 } [get_ports { resetn }]; #IO_L4N_T0_35 Sch=btn[0]
20 set_property -dict { PACKAGE_PIN D19   IOSTANDARD LVCMOS33 } [get_ports { restart }]; #IO_L4P_T0_35 Sch=btn[1]
21
```

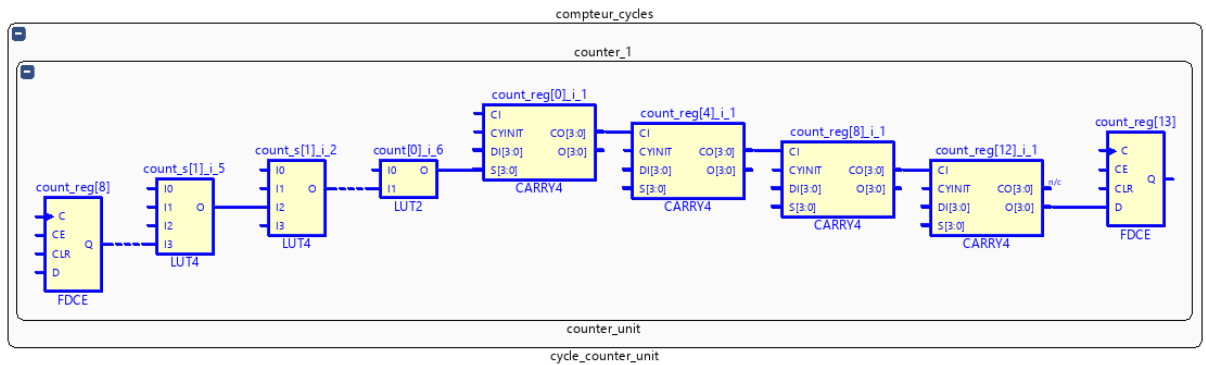
Question 11 :

185	Setup :	0	Failing Endpoints, Worst Slack	5.707ns,	Total Violation	0.000ns
186	Hold :	0	Failing Endpoints, Worst Slack	0.221ns,	Total Violation	0.000ns
187	PW :	0	Failing Endpoints, Worst Slack	3.500ns,	Total Violation	0.000ns

On n'observe aucune violation dans le rapport de timing.

```
191 Max Delay Paths
192 -----
193 Slack (MET) : 5.707ns (required time - arrival time)
194 Source: compteur_cycles/counter_1/count_reg[8]/C
195 (rising edge-triggered cell FDCE clocked by sys_clk_pin (rise@0.000ns fall@4.000ns period=10.000ns))
196 Destination: compteur_cycles/counter_1/count_reg[13]/D
197 (rising edge-triggered cell FDCE clocked by sys_clk_pin (rise@0.000ns fall@4.000ns period=10.000ns))
198 Path Group: sys_clk_pin
199 Path Type: Setup (Max at Slow Process Corner)
200 Requirement: 10.000ns (sys_clk_pin rise@10.000ns - sys_clk_pin rise@0.000ns)
201 Data Path Delay: 4.342ns (logic 1.960ns (45.142%) route 2.382ns (54.858%))
202 Logic Levels: 7 (CARRY4=4 LUT2=1 LUT4=2)
```

Voici le chemin critique correspondant au Max Delay Path, le chemin qui prend le plus de temps à parcourir :



Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5,707 ns	Worst Hold Slack (WHS): 0,221 ns	Worst Pulse Width Slack (WPWS): 3,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 21	Total Number of Endpoints: 21	Total Number of Endpoints: 22

All user specified timing constraints are met.

On peut voir que WNS et WHS sont deux valeurs positives, ce qui confirme l'absence de violations.

Question 12 :

Jusque-là, la fréquence du compteur avait été réduite pour faciliter la simulation et réduire le temps de calcul. Avant de générer le bitstream, on rétablit les bonnes valeurs afin d'obtenir un changement d'état de la LED toutes les 2 secondes. On reprend donc les valeurs du TP2, soit 200 000 000 de coups d'horloges, et on adapte la taille du compteur en conséquence, soit un compteur de taille 28 bits. (Voir vidéo de démonstration)